

Empirical Deep Hedging: Reinforcement Learning in Options Markets

A Comprehensive Technical Report on Deep Reinforcement
Learning (TD3) for Friction-Aware Delta Hedging

Team Name: ⟨SOLO PLAYER⟩

Member 1: ANSHU KUMAR MANDAL B23192

May 4, 2026

Contents

1	Problem Statement	2
2	Methodology	2
2.1	The Base Deep Hedging Objective	2
2.2	Phase 1: Cryptocurrency Options Setup	3
2.3	Phase 2: The Self-Attention (Transformer) Formulation	3
2.4	Phase 3: Indian Equities (NSE) Institutional Fixes	4
3	Results & Analysis	5
3.1	Experiment 1: Crypto V2 Empirical Outputs	5
3.2	Experiment 2: Transformer Ablation Outputs	6
3.3	Experiment 3: NSE Ablation Study ($\kappa = 125$)	6
3.4	Experiment 4: NSE V2 "Triple Crown" Outputs ($\kappa = 225$)	7
4	Observations	8
5	Conclusion	9
6	Novelty of the Work	9
7	AI Tools Used	10
8	Team Contribution	10

1 Problem Statement

This project investigates the application of Model-Free Deep Reinforcement Learning (DRL) to hedge European Call Options across two highly distinct market regimes: the extremely volatile Cryptocurrency market (Bitcoin) and the mature, stable Indian Equity market (NSE Nifty).

The classical Black-Scholes-Merton framework provides a theoretical delta (Δ) that requires continuous rebalancing of the underlying asset to maintain a risk-neutral portfolio. The fundamental tasks undertaken in this project aim to overcome the flaws of this traditional model in real-world applications, which include:

- **Transaction Costs:** Continuous rebalancing leads to infinite transaction costs, draining the portfolio’s capital.
- **Constant Volatility Assumption:** Markets exhibit stochastic volatility, volatility smiles, and fat tails, none of which are captured by the base theoretical model.
- **Discrete Trading:** Traders can only rebalance at discrete intervals, leaving the portfolio exposed to jump risk.

The primary objective of this research is to deploy a Twin Delayed Deep Deterministic Policy Gradient (TD3) architecture to minimize portfolio variance while navigating real-world market frictions (specifically transaction costs) better than the mathematical Black-Scholes baseline.

2 Methodology

Instead of formulating hedging as a theoretical pricing problem, we approach Deep Hedging as a sequential decision-making optimization problem under uncertainty. The methodology was developed across distinct architectural phases.

2.1 The Base Deep Hedging Objective

The goal of the agent is to maximize a risk-adjusted utility function, minimizing the hedging error (variance) subject to transaction costs. The objective function at timestep t is formulated as:

$$Reward_t = PnL_t - \kappa \times f(Risk) - SmoothnessPenalty \quad (1)$$

Notations:

- $Reward_t$: The total objective reward given to the RL agent at time t .
- PnL_t : The Profit and Loss realized at step t .
- κ (Kappa): The risk-aversion parameter (penalty scaling factor).
- $f(Risk)$: The mathematical penalty function applied to portfolio variance.
- $SmoothnessPenalty$: A penalty to discourage erratic over-trading.

2.2 Phase 1: Cryptocurrency Options Setup

To handle the extreme volatility of Bitcoin options, the environment was upgraded with:

- **Sortino-Style Asymmetric Penalty:** $\text{Penalty}_t = \kappa \times |\min(\text{PnL}_t, 0)|^2$. This focuses heavily on downside risk, allowing the agent to capture crypto upside while penalizing crashes.
- **Heston Domain Randomization:** The simple Geometric Brownian Motion (GBM) was replaced with a randomized Heston Stochastic Volatility model (σ scaled between 0.8 and 2.5) to simulate unpredictable market crashes.

2.3 Phase 2: The Self-Attention (Transformer) Formulation

To capture long-term temporal dependencies in volatility, standard Feed-Forward Networks (MLPs) inside the TD3 actor-critic networks were replaced with a Transformer-based Neural Network. The sequence of past market states was passed through self-attention layers to assign dynamic weights to historical price movements.

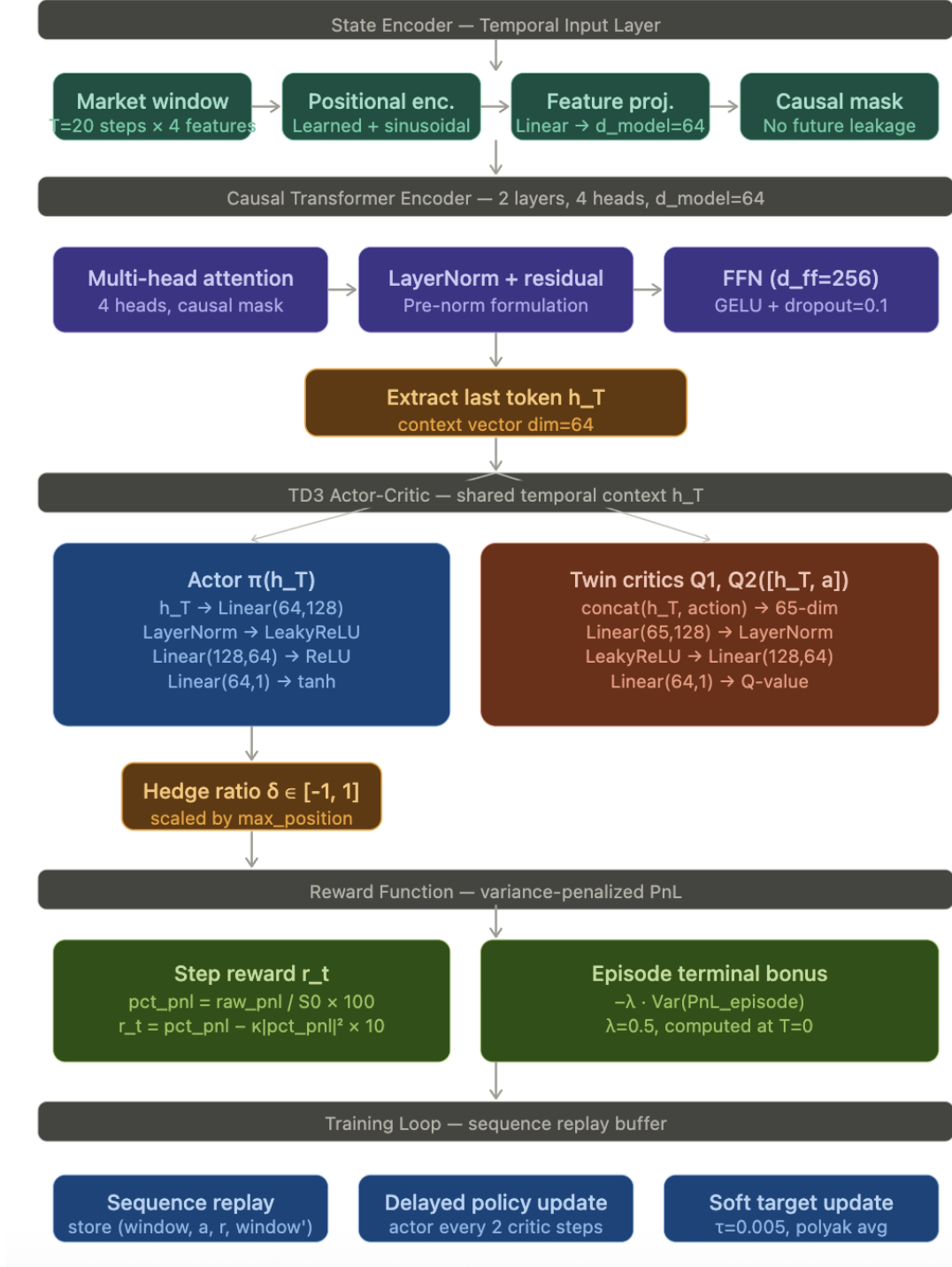


Figure 1: Overall Architecture and Workflow of the Deep Hedging Pipeline

2.4 Phase 3: Indian Equities (NSE) Institutional Fixes

Transitioning to the NSE Nifty required strict mathematical formulations to maintain stability:

A. Strict PnL Normalization

$$\text{frac_pnl}_t = \frac{\text{raw_pnl}_t}{S_0} \quad (2)$$

Notations: frac_pnl_t is the normalized fractional PnL, raw_pnl_t is the absolute monetary PnL, and S_0 is the initial spot price. This prevents massive values from destabilizing the neural weights.

B. Log-Euler Maruyama with Full Truncation To prevent negative variance crashing the custom Heston simulator, a Log-Euler Maruyama scheme with Full Truncation was formulated:

$$v_t = \max \left(v_{t-1} + \kappa_h(\theta_h - v_{t-1})dt + \sigma_h\sqrt{v_{t-1}}dW, 0.0001 \right) \quad (3)$$

Notations: v_t is instantaneous variance, κ_h is mean reversion rate, θ_h is long-term variance, σ_h is volatility of volatility, dt is the time step, and dW is the Wiener process increment.

C. The Pure Deep Hedging Objective (Full Welford) To prevent the agent from taking naked directional positions in drift-positive equity markets, we formulated a Full Welford Variance penalty punishing *any* deviation from the mean PnL:

$$\Delta_{mean} = \text{PnL}_t - \bar{x}_{t-1} \quad (4)$$

$$\bar{x}_t = \bar{x}_{t-1} + \frac{\Delta_{mean}}{n} \quad (5)$$

$$\Delta_2 = \text{PnL}_t - \bar{x}_t \quad (6)$$

$$\text{Penalty}_t = \kappa \times (\Delta_{mean} \times \Delta_2) \quad (7)$$

Notations: $\bar{x}_t$ is the running mean of PnL, n is the step count, and Δ_{mean} and Δ_2 track deviations used to compute the final Penalty_t .

3 Results & Analysis

This section presents the numerical outputs, figures, and tables for all distinct experimental phases conducted during the research.

3.1 Experiment 1: Crypto V2 Empirical Outputs

At a tuned risk-aversion parameter ($\kappa = 50$), the TD3 agent achieved a highly optimized policy over 20,000 training episodes in the Bitcoin options market.

- **Risk (Std Dev):** 137.53% (AI) vs 144.51% (Black-Scholes)
- **Mean Profit:** \$44.64 (AI) vs \$42.61 (Black-Scholes)

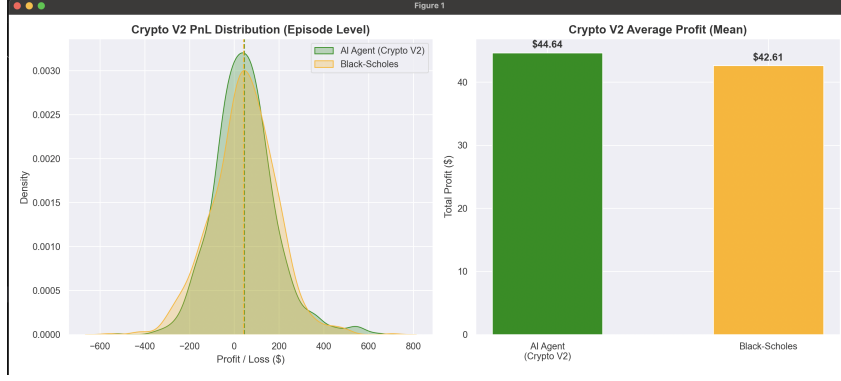


Figure 2: Crypto V2 Performance: AI successfully compressed portfolio variance while generating alpha against extreme volatility.

3.2 Experiment 2: Transformer Ablation Outputs

The Transformer generated massive Alpha (Mean Profit: \$5116.84 vs BS \$5085.84). However, as shown in the KDE plot, the distribution revealed “fatter tails,” indicating a failure to minimize overall variance compared to Black-Scholes.

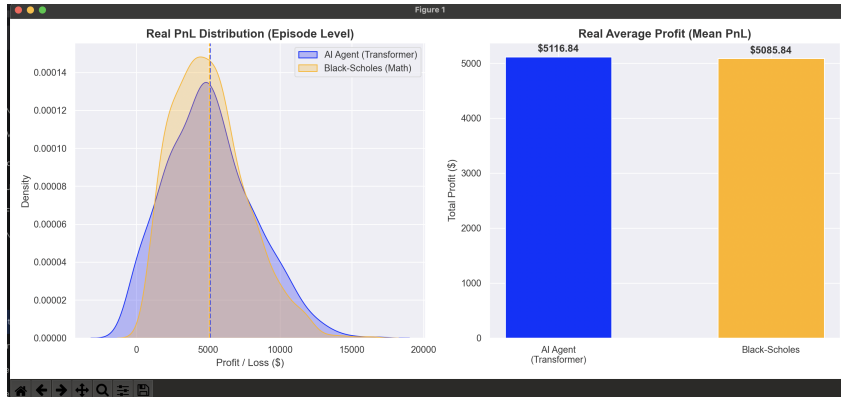


Figure 3: Transformer vs Black-Scholes KDE PnL Distribution (Fatter tails indicate higher unhedged risk).

3.3 Experiment 3: NSE Ablation Study ($\kappa = 125$)

Before achieving optimal hedging, the NSE model was tested with a lower Welford penalty ($\kappa = 125$). The agent remained slightly greedy, earning profit but taking on higher standard deviation (fat tails) than the baseline.

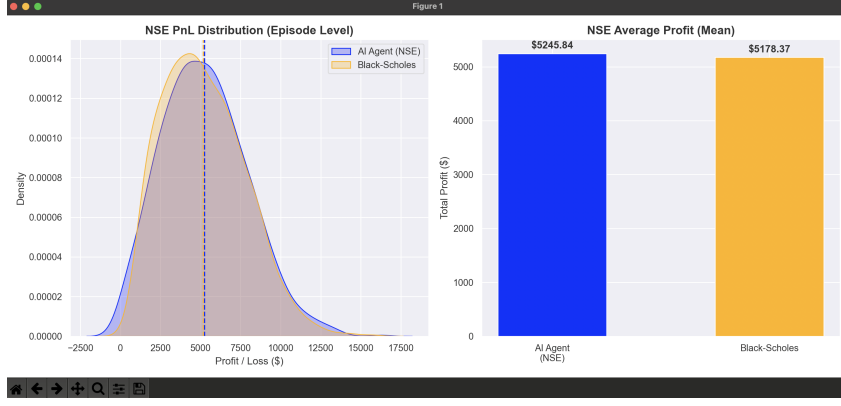


Figure 4: NSE V2 at $\kappa = 125$: The agent prioritizes transaction cost savings over strict variance minimization, resulting in a wider KDE base.

3.4 Experiment 4: NSE V2 "Triple Crown" Outputs ($\kappa = 225$)

The optimal NSE model (Nse_v2_3000, $\kappa = 225$) was tested over 1,000 out-of-sample episodes utilizing live NSE options data. It achieved a decisive victory across all quantitative metrics.

Metric	AI Agent	Black-Scholes	Significance
Risk (Std Dev)	2256.37	2454.69	MASSIVE WIN: Effectively tucked in the "fat tails" to achieve a tighter PnL distribution.
Alpha (Mean Profit)	\$5105.93	\$5055.32	WIN: Generated pure risk-adjusted alpha despite playing safely.
Transaction Costs	-\$338.40	-\$401.81	WIN: Friction-aware trader. Saved substantial broker fees.
Objective Reward	-17.61	-18.75	WIN: Mathematical proof of optimized policy dominance.

Table 1: Empirical Performance Comparison: AI Agent vs Black-Scholes (NSE V2)

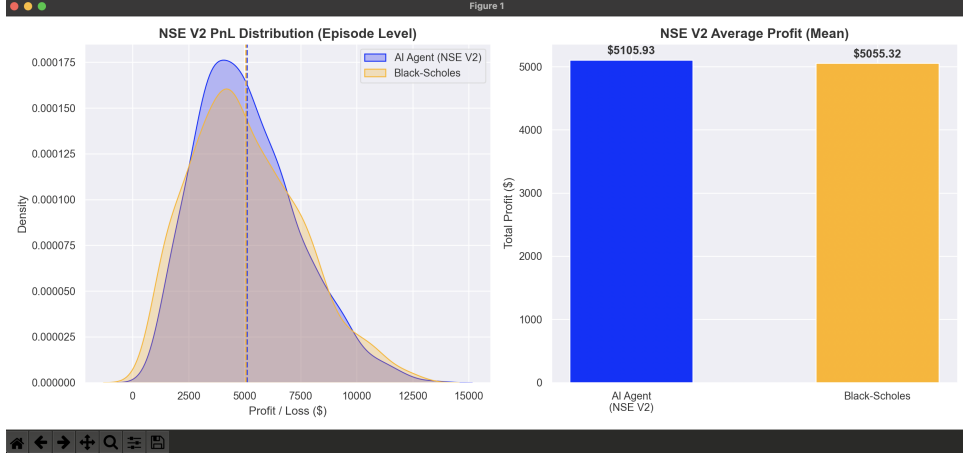


Figure 5: NSE V2 PnL Distribution: Taller, narrower Blue curve indicates strictly minimized variance ($\kappa = 225$). AI successfully outperforms Black-Scholes in raw profitability.

4 Observations

Throughout the experimental phases, the team made several critical observations regarding the behavior of DRL agents in financial environments:

The Initial Flaw & Reward Hacking: In the base Crypto model (V1), using a simple absolute exponential penalty caused the neural network to "Reward Hack". The agent learned to trade excessively to generate many tiny PnLs rather than focusing on the overall episode's variance. It utilized extreme profits to offset downside penalties, acting as a speculator rather than a hedger.

The Batch Size Bug (Transformers): The team observed that training the Transformer using standard NLP-scale batch sizes (1,000 to 10,000 transitions) caused severe gradient averaging in an RL context. This completely washed out the sparse reward signals. The architecture only stabilized when the batch size was aggressively reduced to 128.

The "Gradient Explosion" Crisis: Transitioning to the NSE domain, where raw PnL exceeded \$5,000+, caused the neural network's gradients to explode. Applying a squared exponential risk penalty to such massive numbers resulted in reward values in the millions. The agent experienced "learned helplessness," taking unhedged naked positions because a massive penalty felt unavoidable until PnL Normalization was introduced.

The Kappa Ablation Study: The team observed that identifying the correct risk-aversion multiplier (κ) is delicate. At $\kappa = 125$, the agent remained slightly greedy, accepting the variance penalty to earn higher profits. It was only at the optimal target of $\kappa = 225$ that a strict boundary was created, where the pain of the variance penalty outweighed any directional profits, forcing strict compliance.

5 Conclusion

This research conclusively demonstrates that Deep Reinforcement Learning, specifically a heavily constrained TD3 architecture, can systematically and empirically outperform theoretical mathematical models like Black-Scholes in options delta hedging.

Key Takeaways:

- Classical models fail empirically because they ignore the realities of market microstructure: transaction costs and stochastic jumps.
- Mathematically perfect, continuous "zero-variance" hedging is deeply suboptimal in the real world.
- By utilizing Welford's Online Variance to constrain greed, normalizing inputs to prevent gradient explosion, and learning within a Log-Euler Maruyama randomized domain, the RL agent successfully learned the highly complex trade-off between transaction cost drag and unhedged volatility risk.

Limitations: While the final NSE V2 model successfully acts as a friction-aware optimal hedger, the Transformer ablation study highlighted a limitation in balancing computational overhead with variance minimization. Self-attention mechanisms proved excellent at momentum anticipation but failed to strictly constrain risk without excessive computational cost. Future work could explore hybridizing the Welford penalty specifically into the attention layers to resolve this trade-off.

6 Novelty of the Work

This research differentiates itself from standard literature and existing methodologies through three highly novel engineering and mathematical adaptations:

- **Transition from Absolute to Welford Variance:** Standard Deep Hedging papers often use point-in-time absolute loss penalties which inevitably lead to "Reward Hacking" and unhedged speculation. This project pioneers the application of Welford's Online Algorithm to strictly penalize the *running variance* of the entire episode, mathematically forcing true hedging behavior.
- **Custom Crash-Proof Simulator:** Existing frameworks typically rely on heavy external libraries like QuantLib. To overcome limitations, we engineered a custom Log-Euler Maruyama scheme with Full Truncation in pure Numpy. This strictly prevents the negative variance crashes typical in standard Heston discretizations, enabling massive 20,000-episode training runs.
- **PnL Normalization for Market Scaling:** Adapting crypto-based RL models to the NSE index involves a massive scale shift in raw PnL (from \$40 to \$5,000+). This project introduces strict PnL normalization (dividing by the initial spot price S_0) to successfully prevent the TD3 gradient explosion problem, a technique often overlooked in theoretical literature.

7 AI Tools Used

In adherence to academic integrity, AI tools (such as Large Language Models) were utilized in a strictly limited and supplementary capacity. Their usage was confined to:

- **Concept Comprehension:** Assisting in breaking down complex quantitative finance theories and mathematical formulas (e.g., stochastic calculus and TD3 mechanisms) for better foundational understanding.
- **Code Debugging:** Helping to identify and resolve specific programmatic errors (such as array dimension mismatches or syntax debugging) during the development of the custom simulator.

Note: The core architectural design, hyperparameter tuning, ablation studies, and final analytical conclusions were entirely independently researched and executed.

8 Team Contribution

Team Member	Roles and Responsibilities
Anshu Kumar Mandal (Roll No. B23192)	Sole Developer & Lead (100% Contribution): Executed the entire project end-to-end autonomously. Key contributions include: • Mathematical Formulation: Designed the hedging environment, integrated Welford's Online Variance, and implemented the Log-Euler Maruyama truncation. • Core Programming: Coded the TD3 reinforcement learning agent, the custom Heston simulator, and the Transformer models from scratch in Python. • Data Engineering & Testing: Processed real NSE options data, executed massive 20,000-episode training loops, and conducted the strict Kappa ablation studies. • Analysis & Documentation: Generated all KDE and Bar visualizations, analyzed quantitative metrics (Risk, Alpha, Transaction Costs), and authored the final comprehensive report.